Convolutional Neural Networks for Edge Devices

Advanced Topics in Image Processing, Griffith University

Nathan Burg
Griffith University

Abstract

Convolutional Neural Networks (CNNs) have become the dominant approach for image classification, but their computational requirements often restrict deployment to high-performance hardware and vendor-specific software ecosystems. This paper investigates the feasibility of implementing an end-to-end CNN training and inference pipeline using the Vulkan graphics API as a hardware-agnostic alternative to conventional GPU computing frameworks. A custom Vulkan compute runtime was developed in C, featuring resource management, execution scheduling, and a Lua scripting interface for machine learning workflows. Using this runtime, both a lightweight five-layer CNN and the VGG-16 architecture were implemented and evaluated on the MNIST and CIFAR-100 datasets. Inference performance achieved parity with equivalent PyTorch implementations, demonstrating the correctness of the forward pass implementation. However, training stability remained a significant challenge, with numerical issues limiting model convergence on larger datasets. Despite these limitations, the results demonstrate that Vulkan provides a viable foundation for cross-platform machine learning on edge devices while avoiding dependence on vendor-specific technologies such as CUDA. The work highlights both the opportunities and engineering challenges associated with low-level GPU programming for deep learning applications.

1 Introduction

The domain of digital image processing is among the most diverse in all of computer science, with complex subdomains like compression, generation, and classification. The task of image classification is a unique problem as it is the only task coupled to and directly measured against human perception of images. This is leveraged heavily in user-facing cyber security, in systems like CAPTCHA, where human visual processing cannot be trivially replicated by malicious software [1]—demonstrably, computers are an exceptionally poor analogue of human perception.

Image classification is a domain of machine learning predicated on the 'feature space' of an image. Feature extraction can be understood as a bridge between the suitable representation for human vision, and a representation that is statistically separable between classes. Machine learning classification algorithms exploit this statistical separability using numerical optimisation algorithms, to effectively 'learn' the features associated with a class.

Classical machine learning techniques like the Random Forest [2], k-Nearest Neighbours [3], and Support Vector Machine [4] algorithms rely extensively on careful feature extraction. These algorithms utilise high-dimensional vector representations of an image to classify according to relative alignment in feature space, where the vector is extracted from statistical analysis of an image. Manipulation techniques like convolution serve to make statistical features of an image—such as object edges—more pronounced.

Artificial Neural Networks (ANNs) and the Multi-Layer Perceptron (MLP) are statistical fitting structures that identify common traits between samples of a class using cross-entropy loss and backpropagation [5]. In these simplified models, a 1-dimensional input vector of a feature space is propagated through a series of layers, to then select a class from a 1-dimensional output vector using `softmax` or similar. This is the foundation of all modern machine learning.

Convolutional Neural Networks (CNNs) extensively serve to automate the feature extraction process through deep learning [6]. For small images with high signal-to-noise ratios (SNR) such as MNIST, an MLP is generally sufficient through one-hot encoding of the image itself. For larger images, feature extraction is necessary to identify isolated patterns and improve SNR. CNNs use numerical optimisation to locate the set of convolution kernels that produce the correct feature space, and then use an MLP to classify them, as a two-step machine learning process.

The training and inference process of CNNs are generally very computationally expensive, and require expensive GPU hardware to deploy practically. Vendors of these devices each provide their own proprietary solution to General-Purpose GPU programming (GPGPU), and in the specific case of machine learning the industry is largely centralised around the NVIDIA CUDA ecosystem.

We propose an experimental solution that relies on Vulkan, which is a computer graphics API that can be leveraged for GPGPU programming through headless 'compute shaders.' Vulkan is designed for maximum hardware compatability, but also exposes the full functionality of various GPU architectures through a rich set

of extensions [7]. Vulkan compatibility extends beyond consumer desktop GPUs, and also includes mobile phone chipsets, edge devices like the NVIDIA Jetson, and single-board computers like the Raspberry Pi. Our goal is to deliver a prototype of a complete GPU machine learning pipeline for Convolutional Neural Networks using the Vulkan API, and demonstrate the engineering challenges of developing such a pipeline under significant time constraints.

2 Literature Review

2.1 Image Classification

In order to approximate the learning effects of human vision, algorithmic image classification generally requires a large repository of sample images. This repository must be labeled with a ground-truth baseline, which establishes the underlying principle of machine learning and image classification. Principally, these techniques do not have the same subjective visual understanding as humans, but instead use statistical approximations of what feature vector is most geometrically similar to manually labeled target.

Image classification is entirely predicated on clean repositories of images, which itself is subject to significant quality control requirements. Using algorithms to classify images constitutes a substantial portion of the literature, but it is also important to consider the difficult problem of data collection and labeling, and the importance of clean data particularly in the area of highly expressive machine learning models.

A complete image classification pipeline has three core components, being the image repository for training, the feature extraction algorithm, and the feature space classifier. This specific coupling is the general foundation of all classification pipelines in machine learning.

2.1.1 Image Repositories

As image classification as a concept is almost entirely predicated on high-quality training data, images should be of consistent quality across classes, correctly labeled, and ideally of high signal-to-noise ratio to prevent shortcut learning [8]. A low-quality dataset leads to poor generalisation in trained models—because many modern training sets are effectively 'black boxes' in terms of content with image counts in the millions, consistent methodological rigour of collection and labeling should be practiced.

The MNIST dataset is common for simple image classification tasks, and consists of 60000 greyscale images at a resolution of 28×28 [6]. The subject of the images is handwritten numerical digits. The simplicity of the images in this dataset permit usage of the image itself as a feature vector—as the input size of 784 is small enough to be tractable—making MNIST a common target for MLP-based classifiers.

The CIFAR family of datasets, including both CIFAR10 and CIFAR100, are more complex images that are harder to separate in feature space than MNIST. CIFAR images are 32×32 RGB depicting real subjects across several mutually exclusive categories, such as dog, deer, truck and ship [9]. As this dataset consists of RGB images, the size of the input vector increases dramatically to 3072, which significantly reduces the SNR in feature space. It is generally more common for CIFAR classification to be solved with convolution-based networks, as the 3 channels represent a tensor that maps directly to CNN architecture, and the feature extraction of the convolution layers can more effectively filter noise.

ImageNet is considered the standard dataset in computer vision research. The complete set contains over 14 million images across more than 20000 classes at varied image resolutions [10]. An extensive preprocessing pipeline is needed for this dataset to normalise image resolution to a standard square appropriate for standardised feature extraction. A variant of ImageNet called ImageNet-1k or ILSVRC2017 is very common in model benchmarks, where breakthrough models like AlexNet and VGG were developed and deployed specifically to classify on this dataset [11].

2.1.2 Feature Extraction

Feature extraction is fundamentally a signal processing problem. Images are easily identifiable to humans, and perhaps counterintuitively the visual signatures that allow this recognition are effectively noise in terms of algorithmic processing. Feature extraction maps high-noise image data into high-signal 'hints' about a certain feature of the image, which can include edge detection, colour histograms, or frequency-space transforms. The primary objective of feature extraction is to reduce the high-frequency image into a statistically separable feature vector.

Earliest feature extraction techniques, such as Otsu's thresholding, use a colour histogram for the image to apply a binary threshold and separate a subject from the background [12]. The Sobel filter and Canny edge detection use convolution filters to isolate object edges [13, 14]. As examples of feature extraction, these techniques correctly amplify the signal of desired features, but do not reduce the dimensionality of feature space.

Advanced techniques like Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transforms (SIFT) apply several convolution filters in sequence to extract localised colour histograms for discrete segmentations of an image [15,16]. These techniques differ in the shape of the output data, which governs its utility—SIFT produces a list of points-of-interest for an image, while HOG produces a fixed-dimension distribution of edge normal vectors. The feature vector of HOG is invariant while SIFT produces a list of relevant edges. For HOG specifically, this feature space is highly separable using machine learning, and is very useful for classification problems.

2.1.3 Machine Learning

Machine learning is applied numerical optimisation for statistical prediction problems. For image classification, a model must be optimised to separate classes using geometric and statistical difference between feature space vectors. This can be achieved with modern deep learning networks, but algebraic or probabilistic techniques—often called 'classical machine learning'—are also highly effective and generally more computationally efficient for classification of highly separable feature vectors. The Support Vector Machine (SVM) is an example of classical machine learning, which treats a feature vector as a high-dimensional geometric object and classification as a convex quadratic optimisation problem [4]. The output of an SVM is a max-margin hyperplane for the decision boundary, which is trivial to compare during classification.

Random Forest (RF) classifiers observe the feature vector as a set of subspace elements [2]. Decision trees for an input set are formed from localised subsets of the feature vector, with an independent tree for each. During classification, an input feature vector is segmented and evaluatied for each sub-feature, and the final output is the majority decision of each sub-feature decision tree.

K-Nearest Neighbours (kNN) is an instance-based algorithm that classifies images by geometric distance in feature space [3]. Being instance-based, kNN has no training process, and trivially returns multiple images that are 'most similar' to the input feature vector. The data stored for this algorithm is far higher than SVM or RF, as each training image is retained and ranked lazily. The benefit of this is total dynamic classification where images can be added or removed individually, which is impossible in SVM and RF.

Each of these methods, while effective classifiers, rely on statistically effective feature extraction, which is an unsolved problem and is not consistent across domains. Modern techniques like Convolutional Neural Networks implicitly couple feature extraction with classification, forming a complete classification solution [6]. Consequently, classical machine learning is less utilised in modern image classification literature.

2.2 Deep Learning

Deep learning and neural networks are a class of statistical prediction models that encode arbitrary nonlinear functions in high-dimensional latent space. The model itself is constructed of several layers of high-dimensional vectors called 'layers'. Propagating an input vector through these layers involves two steps—the transformation, and the activation. The first stage applies the model 'weights', which are a linear transformation in feature space that weight features toward an output signal. The activation function then applies a nonlinear transform, such as ReLU, to effectively distort the linear input signal into nonlinear feature space. In the final layer, the `softmax` function is applied to identify the predicted class using the non-normalised 'logits.' [5]

The training process of a neural network is first predicated on a loss function, which effectively represents the magnitude of misprediction by the neural network, which becomes a gradient in vector space. This vector is then propagated backwards through the network and the layer weights are updated according to this gradient. Multiple functions must be simultaneously approximated in neural network training, so the maximum difference or 'learning rate' for this weight perturbation is vanishingly small. The desired outcome is slow convergence toward an approximate function that can classify data not included in the training set.

2.2.1 Multi-Layer Perceptron

The Multi-Layer Perceptron (MLP) is a standard formulation of a fully-connected neural network. This form operates exclusively on one-dimensional input and output vectors, which poses structural limitations for image classification problems. Despite retaining the complete input signal, there is no consideration of spatial coupling inherent to the architecture, meaning the topology of the image is destroyed—this also means manual feature extraction of the image must be performed. While an MLP can effectively classify a feature vector representation of an image, classical machine learning techniques can do so more efficiently. This directly motivates a coupling of feature extraction and classification for deep learning image classification to be tractable.

2.2.2 Convolutional Neural Networks

Convolutional Neural Networks extend MLPs by propagating layers in higher dimensions. CNNs are particularly suited to image recognition, as they can be understood as a unified pipeline of both feature extraction and

classification. This is achieved by representing the early layers as 3D tensors instead of 1D vectors, encoding both spatial information and colour channel information. The linear transform is then a convolution for each element of this tensor, where the convolution kernel is the weight for that layer. This tensor is then downsampled along its spatial dimensions, and subsequent convolutional layers utilise expanded kernel dimensions to increase channel depth. This repeats until the tensor is near-1-dimensional, where it then becomes the input for an MLP-like classifier [6].

LeNet-5 The original formulation of the Convolutional Neural Network was designed for handwritten digit and symbol classification, specifically motivated by the topological collapse of MLPs for this problem [6]. To preserve spatial structure, LeNet introduced the structural template of modern CNN architectures by stacking successive convolution and sub-sampling blocks that terminate to fully connected layers. While effective for low-resolution digit recognition compared to an MLP, the architecture lacked the computational scaling capacity and depth required to process and classify more complex images.

AlexNet This formulation optimises the basic CNN by introducing the Rectified Linear Unit (ReLU) to CNN architectures [11]. The ReLU diverges from the standard artificial neuron design, which traditionally relies on sigmoid or `tanh` activation producing normalised values between 0 and 1 [17]. ReLU reduced training time on CIFAR-10 by about 85%, allowing the model to scale to larger input tensors, and more images [11]. Consequently, AlexNet superseded all available models at the time until sufficient GPU hardware development permitted more advanced formulations.

VGG The VGG architecture investigates the effect of CNN network depth on classification performance [18]. By using a uniform convolution kernel size—the smallest possible at 3×3—and adding more convolution layers, VGG introduces stacked convolution and the effective receptive field. This homogeneous stacking yields both increased nonlinearity through additional ReLU activations, and reduced the overall parameter count relative to larger single-layer filters. The VGG-16 formulation, applied to the ImageNet dataset, supplanted AlexNet in classification accuracy.

Structurally, VGG is limited by its computational cost. The additional layers substantially increase training time and memory overhead, making it impractical in many applications. Furthermore, increasing the model depth even further has a boundary where performance begins to degrade, which is attributed to the vanishing gradient problem. As a gradient passed through backward activations in training, the gradient becomes smaller with each layer, where it is eventually extinguished by quantisation error in floating point division.

ResNet While VGG established that architectural depth directly correlates to feature abstraction performance, the vanishing gradient problem cannot be suppressed by conventional CNN architectures. The Residual Network (ResNet) was introduced to redefine the learning objective of deep convolutional layers by introducing identity skip connections that allow the layers to learn a residual mapping of activation outputs [19]. Rather than forcing a sequence of layers to approximate an absolute mapping $H(x)$, ResNet structures the convolutional block to learn a residual mapping, formalised as $F(x) = H(x) - x$. The original input tensor x bypasses the weight layers entirely through a shortcut path and is recombined through element-wise addition. This yields the final block activation:

$$H(x) = F(x) + x \tag{1}$$

which fundamentally alters the mechanics of backpropagation:

$$\frac{\partial H(x)}{\partial x} = \frac{\partial F(x)}{\partial x} + 1 \tag{2}$$

The constant +1 term ensures that even if the gradient through the convolutional layers ($\frac{\partial F(x)}{\partial x}$) approaches zero, the error signal can propagate backward across the addition operator completely unobstructed. By establishing these checkpoints for gradient flow, ResNet effectively mitigates gradient attenuation, enabling the stable convergence and higher accuracy of deep networks (such as ResNet-50 and ResNet-152) that far surpass the depth limitations of VGG.

2.3 Edge Computing

2.3.1 Definition and Application

Edge computing in the context of computer vision is the migration of compute workloads from centralised cloud servers directly to the physical periphery where data is collected. For many applications, connection stability, latency constraints, or infrastructure may limit the availability and utility of centralised computing.

This environment is constrained primarily by Size, Weight, Power and Cost (SWaP-C). Edge topologies typically operate on constrained power budgets and possess limited thermal dissipation capabilities, making them hostile to the massive parameter counts and floating-point operations inherent to state-of-the-art CNNs [20]. Consequently, deploying deep learning models to the edge necessitates a strict focus on compute efficiency, architectural compression, and highly optimised hardware execution.

2.3.2 Vulkan

Vulkan is a modern graphics API designed to operate on low-level GPU primitives, which has the secondary benefit of near-universal GPU compatibility [7]. A common usage in General-Purpose GPU (GPGPU) programming is cross-platform massively-parallel computation using Vulkan—unlike previous abstractions like OpenGL ES, Vulkan directly enables arbitrary command buffer submission for headless execution of user-defined compute shaders. Several frameworks currently exist for cross-platform machine learning using Vulkan, including backends of bleeding edge libraries like `llama.cpp`, Vulkan Kompute, and NCNN.

The structural significance of Vulkan is threefold:

1. For high-powered devices, Vulkan enables precise, low-level GPU control that can match or exceed the performance of industry-standard, high-level abstractions. Device specific features are supported through Vulkan extensions to push performance even further.
2. The proprietary vendor lock-in of NVIDIA's CUDA ecosystem is completely bypassed, permitting efficient parallel execution across diverse hardware architectures
3. An overwhelming majority of modern edge devices, including smartphones and single-board computers, feature high-performance Vulkan drivers capable of executing pre-compiled SPIR-V bytecode with deterministic precision.

3 Methodology

The primary artefact of this paper is an end-to-end Vulkan compute and machine learning runtime, with a structurally complete implementation of the VGG convolutional neural network architecture. This software is composed of several layers of abstraction for practical extensibility.

3.1 Vulkan Compute Runtime

The primary driver for Vulkan GPGPU programming for this experiment is a queue-based GPU program dispatch runtime. The program enables the loading of encoded images such as `.gif .png .jpg` through `stb_image.h`, binary images, and a custom paginated image atlas format for bulk operations. The header `image.h` provides extensive primitives for allocating resources, defining upload and download paths, and modifying the execution queue. This architecture allows composable execution of GPU programs through a rich Lua scripting layer. The program is written in the C programming language, targeting a local Vulkan driver with minimal dependencies (Lua, LunarG Vulkan SDK, shaderc, `stb_image.h`, `renderdoc_app.h`).

3.1.1 Resource Management

The program exposes a small API of resource management functions for various operations. For host (CPU-side) memory, images and image atlases are explicitly created from their respective initialisation functions. In the case of image atlases, images created from an atlas are simply views where the atlas retains ownership of the memory. For device (GPU-side) resources, individual buffers and images usable in programs are allocated by their respective allocation functions, and an integer index is returned corresponding to the bindless array index of the resource, which is usable within GPU programs. Device objects are managed by internal allocators within the `gpu_t` struct, allowing the GPU state to be easily reset without manual resource cleanup.

Loading GLSL shaders is automatically managed, as all pipeline and descriptor layouts are unified through the bindless resource system. Loading a shader will invoke shaderc to compile the GLSL into SPIR-V, and a new `VkPipeline` object will be created and returned as a `gpu_program_t`. GLSL compute shaders have a 3-dimensional local workgroup that may be defined when loading a GPU program. Setting this value scales dispatch resolution with local workgroup—for example, to process a 100×100 image, a loaded shader with a local workgroup size of 10×10 may be invoked with `dispatch(100,100)`, and the dispatch size will be scaled in integer workgroup tiles automatically. These GPU program objects may be passed to the execution queue, allowing both single-time, repeating, and sequential execution of GPU programs.

3.1.2 Execution Queue

The primary data structure for submitting GPU work is the execution queue. The `gpu_program_t` primitive may be passed to the execution queue, adding a new stage. Creating a GPU execution stage (`img_gpu_add_stage`) returns an index within the internal stage allocator, then stage data may be passed (`img_gpu_add_stage_data` and `img_gpu_set_stage_data`) using the stage index as an argument. Stage data is a 4-byte aligned serial buffer, where each 'add' operation bumps the offset index. Both individual values and arrays, such as convolution kernels, can be added easily. Memory for stage data is automatically managed within the internal allocator of the queue.

Uploading and downloading data is automatically managed through the execution queue. Calling the `img_gpu_upload` and `img_gpu_download` functions with both a host and device pointer will automatically allocate a staging buffer, and schedule transfer of resources with correct ordering at dispatch time. For immediate uploads and downloads, such as between submissions, the `_now` versions of these functions will allocate temporary staging buffers and immediately perform the transfer.

After defining the allocations, stages, stage data, and transfers, the `img_gpu_dispatch` function may be called to record the command buffer and submit the work queue. This function will upload the input data, apply stage data as GPU push constants, execute the stages, then download the data. All memory barriers and texture layout transitions are automatically managed.

For repeated dispatches, upload and download pointers can be changed with the `map_device` and `map_host` functions to change what host resource to upload, or the device resource to download. Stage data may also be modified with `img_gpu_set_staging_data`, and is indexed by the 4-byte index of the data to be changed. This is useful for modifying program behaviour between executions—such as constant scalars, convolution kernels or branch flags—without loading a new program.

3.1.3 Lua Scripting API

The primary interface for driving the runtime is a Lua scripting layer, exposing all native functions while abstracting resource types. This scripting layer allows the main program to retain minimal functionality as a bare interface for executing GPU programs, and any GPU work may be executed for any use case. For demonstration, two image classification implementations were delivered through the Lua interface—a HOG-SVM classifier, and a composable Convolutional Neural Network inference and training API.

3.2 Machine Learning Environment

To implement VGG within this Vulkan runtime, a set of utility functions were developed to map the neural network architecture to established GPU primitives. A comprehensive set of utilities were defined for simple deployment of CNNs, similar to the PyTorch workflow. The Lua module `cnn.lua` includes simple functions for integrating CNNs with the standard image pipelines, such as converting RGB images into usable tensor formats, generating CNNs from high-level architecture tables, and loading model weights from disk. The core loop of training and inference is user-defined, but integrates smoothly with the standard image pipeline, allowing simple upload, download, and validation to suit user requirements.

3.2.1 Data Processing

The various datasets used in this experiment have no standardised format. For the target datasets discussed in Section 3.2.3, the `.parquet` file format was preferred as a baseline. A Python script was used to convert `.parquet` files into the binary atlas format recognised by the Vulkan runtime. The atlas format allows native pagination of data, which saves memory and improves read time when loading from disk, which is useful for this high-throughput task.

3.2.2 Feed-Forward

To verify the functionality of the primitives of a convolutional neural network—like the convolution layers, pooling layers, and the infrastructure for classification loops—the forward pass of VGG can be implemented in isolation. For the specific combination of VGG-16 and CIFAR-100, existing model weights are readily available to validate functionality. A short Python script is used to extract the weights and save them in a binary format supported by the Vulkan runtime. Batch normalisation is not supported in the program, so the script also folds these parameters into the weights. The Lua environment can load these weights after initialisation of the CNN.

3.2.3 Backpropagation

Implementing VGG with no supporting libraries is not trivial. As VGG is typically trained on ImageNet-1k, even early validation tests will consume a large portion of available time. Using leaner architectures on smaller

datasets will provide a low-cost baseline for more complicated models. For this purpose, both a simple 5-layer CNN, and a full implementation of VGG-16 will be tested across two datasets. In terms of machine learning primitives, the ReLU activation function and Adam optimiser are used, with a batch size of 1 and learning rate of `1e-3`.

MNIST The first test is a simplified 5-layer convolutional neural network on the MNIST dataset. MNIST contains 60000 28×28 images of handwritten digits, and is comparatively simple for even MLPs to resolve correctly. For this reason, a simple CNN will be faster to train and easier to observe and validate correct behaviour.

Operator	Input Dimensions	Output Dimensions
Input	$28 \times 28 \times 1$	–
Conv3	$28 \times 28 \times 1$	$28 \times 28 \times 64$
Pool	$28 \times 28 \times 64$	$14 \times 14 \times 64$
Conv3	$14 \times 14 \times 64$	$14 \times 14 \times 128$
Pool	$14 \times 14 \times 128$	$7 \times 7 \times 128$
Conv3	$7 \times 7 \times 128$	$7 \times 7 \times 256$
Pool	$7 \times 7 \times 256$	$4 \times 4 \times 256$
FC	$4 \times 4 \times 256$	$1 \times 1 \times 256$
FC	$1 \times 1 \times 256$	$1 \times 1 \times 10$
Softmax	$1 \times 1 \times 10$	–

Table 1: CNN-5 architecture for MNIST. Five total weight layers. ReLU lines omitted.

CIFAR-100 The second test is VGG-16 on the CIFAR-100 dataset. The resolution of CIFAR-100 is high enough to map to VGG-16 without truncation, and the low-SNR of the images provide a failure mode for the simplified model that may be demonstrably surpassed by VGG-16.

Operator	Input Dimensions	Output Dimensions
Input	$32 \times 32 \times 3$	–
Conv3	$32 \times 32 \times 3$	$32 \times 32 \times 64$
Conv3	$32 \times 32 \times 64$	–
Pool	$32 \times 32 \times 64$	$16 \times 16 \times 64$
Conv3	$16 \times 16 \times 64$	$16 \times 16 \times 128$
Conv3	$16 \times 16 \times 64$	–
Pool	$16 \times 16 \times 128$	$8 \times 8 \times 128$
Conv3	$8 \times 8 \times 128$	$8 \times 8 \times 256$
Conv3	$8 \times 8 \times 256$	–
Conv3	$8 \times 8 \times 256$	–
Pool	$8 \times 8 \times 256$	$4 \times 4 \times 256$
Conv3	$4 \times 4 \times 256$	$4 \times 4 \times 512$
Conv3	$4 \times 4 \times 512$	–
Conv3	$4 \times 4 \times 512$	–
Pool	$4 \times 4 \times 512$	$2 \times 2 \times 512$
Conv3	$2 \times 2 \times 512$	–
Conv3	$2 \times 2 \times 512$	–
Conv3	$2 \times 2 \times 512$	–
Pool	$2 \times 2 \times 512$	$1 \times 1 \times 512$
FC	$1 \times 1 \times 512$	–
FC	$1 \times 1 \times 512$	–
FC	$1 \times 1 \times 512$	$1 \times 1 \times 100$
Softmax	$1 \times 1 \times 100$	–

Table 2: VGG-16 architecture for CIFAR-100, 16 total weight layers.

ImageNet-1k The final objective of this runtime is to implement VGG-16 to train a model using ImageNet-1k, to achieve parity the original formulation of VGG-16. This model contains many orders of magnitude more parameters, and is not practical for pipeline validation, however due to its size it is a highly effective benchmark for GPU throughput and enables measurement against state-of-the-art implementations.

Operator	Input Dimensions	Output Dimensions
Input	$224 \times 224 \times 3$	–
Conv3	$224 \times 224 \times 3$	$224 \times 224 \times 64$
Conv3	$224 \times 224 \times 64$	–
Pool	$224 \times 224 \times 64$	$112 \times 112 \times 64$
Conv3	$112 \times 112 \times 64$	$112 \times 112 \times 128$
Conv3	$112 \times 112 \times 64$	–
Pool	$112 \times 112 \times 128$	$56 \times 56 \times 128$
Conv3	$56 \times 56 \times 128$	$56 \times 56 \times 256$
Conv3	$56 \times 56 \times 256$	–
Conv3	$56 \times 56 \times 256$	–
Pool	$56 \times 56 \times 256$	$28 \times 28 \times 256$
Conv3	$28 \times 28 \times 256$	$28 \times 28 \times 512$
Conv3	$28 \times 28 \times 512$	–
Conv3	$28 \times 28 \times 512$	–
Pool	$28 \times 28 \times 512$	$14 \times 14 \times 512$
Conv3	$14 \times 14 \times 512$	–
Conv3	$14 \times 14 \times 512$	–
Conv3	$14 \times 14 \times 512$	–
Pool	$14 \times 14 \times 512$	$7 \times 7 \times 512$
FC	$7 \times 7 \times 512$	$1 \times 1 \times 4096$
FC	$1 \times 1 \times 4096$	–
FC	$1 \times 1 \times 4096$	$1 \times 1 \times 1000$
Softmax	$1 \times 1 \times 1000$	–

Table 3: VGG-16 (Configuration D) architecture for ImageNet-1k 224×224, 16 total weight layers.

3.3 Validation and Debugging

As GPU programs do not expose typical tools like logs and debuggers in the general sense, certain tools are available to trace errors in the program. As this application consists of many inter-operating components, utilising these tools to prevent unsolvable errors is prerequisite to the success of the project. Two primary tools will be utilised at different levels of abstraction:

1. Vulkan validation layers are a driver-level feature that identify GPU errors, such as API misuse or GPU driver crashes. As GPU crashes generally lead to an untraceable `SIGSEGV`, validation layers provide useful indication of common failure modes. Particularly, GPU crashes are triggered by Undefined Behaviour (UB) on the GPU, such as out-of-bounds memory reads or writes, division by zero, and infinite loops.

2. RenderDoc is a standard graphics debugger used for capturing execution stacks of computer graphics programs. RenderDoc provides a header-only API for capturing frames within headless applications, allowing frame capture for low-level debugging of individual resources at different pipeline stages. This allows us to track resources over time, verify GPU program arguments are correct, and verify resource bindings are used correctly.

Other standard debugging tools like GDB and Valgrind will also be utilised where necessary.

4 Results

Implementation of this pipeline demonstrates the rigorous engineering required for these problems. An inference pipeline for VGG-16 was successfully implemented, however the training pipeline introduced significant numerical instability. For VGG-16, inference could achieve parity with equivalent PyTorch pipelines, but training retained a limit of 70% accuracy and 1.0 loss in the best attempts.

4.1 Inference

Inference was implemented successfully for classification on the CIFAR-100 dataset. Model weights were generated through an external Python script, and when loaded to the VGG-16 architecture an accuracy of 0.995 on the training set and 0.7 on the testing set was consistently achieved. This indicates that the forward pass is functioning correctly.

4.2 Training

Model training is significantly more complex than inference. The varying resource shapes require many permutations of similar GPU programs, which makes consistent implementation across dimensions exceedingly difficult. Several bugs in the program were discovered throughout this implementation, and particularly the initial image processing design was revealed to be cumbersome for CNN training tasks.

Earliest training experiments yielded extremely poor results, consequent to several compounding design flaws and mathematical errors. When simplifying the dataset and CNN architecture, plausible results were recovered, and furthermore when training on small subsets of image data the model could correctly fit these images. This indicates that while the model is capable of learning beyond random chance, there are significant parameterisation errors that prevent late-stage fine tuning.

Dataset/Architecture	**Number of Images**	**Max. Accuracy**
CIFAR-100, VGG-16, 100 classes	50000	0.03
	10000	0.20
	500	0.70
	<50	1.00
MNIST, CNN-5, 10 classes	60000	0.70
	10000	0.70

Table 4: Session training results. Max. Accuracy indicates rolling accuracy on training data (self fitting) per epoch.

For the MNIST model, the weights were recorded and validated on a test set. For held-out data, this model achieved similar performance around 0.7, which suggests reasonable generalisation but poor overall performance for MNIST classification expected by CNNs of any size.

4.3 Performance

The execution queue and resource management model ensured near-deterministic performance during program execution, however development resources were allocated to correctness and functionality, therefore performance is not representative of what is possible in this context. There was very little optimisation of GPU programs, or the execution queue itself, and batch operations were not implemented at all.

Configuration	**Time per Image (ms)**
CNN-5, inference	0.5 ms
CNN-5, training	0.9 ms
VGG-16, inference	14 ms
VGG-16, training	35 ms

Table 5: Execution time per-image in inference and training. All measurements are GPU hardware timers, recorded on an RTX 2080 throttled to 150 W total power.

5 Discussion and Future Work

The implementation, experiments and iterative refinements highlight the engineering constraints of edge-oriented GPGPU development. While Vulkan permits execution across diverse hardware, the absence of high-level libraries necessitates manual implementation of backpropagation. This algorithm has no universal solution and is highly sensitive to numerical stability and implementation details. To ensure success of this experiment in the future, foundational backpropagation primitives should be regarded with higher importance. This implementation should be better understood as a bottom-up design, where instead of starting at VGG and working backwards, the runtime should yield a general-purpose autograd environment that VGG may be trivially implemented within.

As the program was initially designed as a single-execution image processing tool, retroactively fitting repeat execution forced trade-offs in terms of API consistency and stability, and the Lua scripting environment was often detractive in development velocity as the dynamic typing system introduces friction with delicate GPU primitives.

Considering this, improving this program primarily consists of designing for machine learning operators, and not images. In the general case, real images are secondary to machine learning implementations, and for resource-constrained edge devices this weighting must be well understood.

6 Conclusion

This paper has demonstrated the feasibility of a headless, Vulkan-based machine learning runtime for edge devices. By moving compute workloads to a platform and architecture agnostic graphics API, we have bypassed

vendor lock-in while maintaining performance and massively parallel execution. Although training stability in the prototype requires further refinement, the inference implementation achieves parity with established frameworks. This work establishes a baseline for future research in accessible, cross-platform deep learning on constrained edge hardware.

References

[1] L. Von Ahn, M. Blum, N. J. Hopper, and J. Langford, "CAPTCHA: Using Hard AI Problems for Security," in *Advances in Cryptology — EUROCRYPT 2003* (G. Goos, J. Hartmanis, J. Van Leeuwen, and E. Biham, eds.), vol. 2656, pp. 294–311, Berlin, Heidelberg: Springer Berlin Heidelberg, 2003. Series Title: Lecture Notes in Computer Science.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, Oct. 2001.

[3] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, pp. 21–27, Jan. 1967.

[4] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, Sept. 1995.

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, Oct. 1986.

[6] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, pp. 2278–2324, Nov. 1998.

[7] Khronos Group, "Vulkan 1.4 Specification," 2026.

[8] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, Nov. 2020. arXiv:2004.07780 [cs.CV].

[9] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," 2009.

[10] J. Deng, W. Dong, R. Socher, L.-J. Li, Kai Li, and Li Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, (Miami, FL), pp. 248–255, IEEE, June 2009.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, pp. 84–90, May 2017.

[12] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, pp. 62–66, Jan. 1979.

[13] I. Sobel, R. Duda, P. Hart, and J. Wiley, "Sobel-feldman operator," *Preprint at https://www. researchgate. net/profile/Irwin-Sobel/publication/285159837. Accessed*, vol. 20, p. 30, 1968.

[14] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, pp. 679–698, Nov. 1986.

[15] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 1, (San Diego, CA, USA), pp. 886–893, IEEE, 2005.

[16] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, pp. 91–110, Nov. 2004.

[17] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ICML'10, (Madison, WI, USA), pp. 807–814, Omnipress, 2010.

[18] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," Apr. 2015. arXiv:1409.1556 [cs.CV].

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Dec. 2015. arXiv:1512.03385 [cs.CV].

[20] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 637–646, Oct. 2016.